

MARAN PMS — Módulo Eventos

PROMPT: Desglose asistentes + Día semana en Hoja de Función + Campos por área

CAMBIO 1 — Desglose de asistentes: adultos / jóvenes / niños / total

A — Migración de base de datos

Agregar tres columnas nuevas a la tabla `events` :

```
ALTER TABLE events
  ADD COLUMN IF NOT EXISTS attendees_adults    integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS attendees_youth     integer NOT NULL DEFAULT 0,
  ADD COLUMN IF NOT EXISTS attendees_children integer NOT NULL DEFAULT 0;
```

La columna `attendees` existente pasa a ser el **total calculado** (suma de los tres).

B — Schema Drizzle (`shared/schema.ts`)

Agregar las tres columnas al schema de la tabla `events` :

```
attendeesAdults: integer("attendees_adults").default(0),
attendeesYouth:  integer("attendees_youth").default(0),
attendeesChildren: integer("attendees_children").default(0),
```

C — Schema Zod del form (`eventFormSchema`)

Reemplazar el campo `attendees` por los cuatro campos nuevos:

```
// Antes:
attendees: z.number().min(1, "Debe tener al menos 1 asistente"),

// Después:
attendeesAdults: z.number().min(0).default(0),
```

```

attendeesYouth:      z.number().min(0).default(0),
attendeesChildren: z.number().min(0).default(0),
attendees:           z.number().min(1, "Debe haber al menos 1 asistente en total"),
// attendees se calcula automáticamente, no se edita manualmente

```

D — Form de nuevo evento y form de edición

Reemplazar el `FormField` de `attendees` por cuatro campos en una grilla 2x2, con el total calculado automáticamente:

```

{/* Grilla de asistentes */}
<div className="col-span-2 space-y-2">
  <FormLabel>Asistentes</FormLabel>
  <div className="grid grid-cols-2 gap-3">

    {/* Adultos */}
    <FormField
      control={eventForm.control}
      name="attendeesAdults"
      render={({ field }) => (
        <FormItem>
          <FormLabel className="text-xs text-muted-foreground">Adultos</FormLabel>
          <FormControl>
            <Input
              type="number"
              min={0}
              {...field}
              onChange={(e) => {
                field.onChange(parseInt(e.target.value) || 0);
                recalcTotal();
              }}
              data-testid="input-attendees-adults"
            />
          </FormControl>
        </FormItem>
      )}
    />

    {/* Jóvenes */}
    <FormField
      control={eventForm.control}
      name="attendeesYouth"
      render={({ field }) => (
        <FormItem>
          <FormLabel className="text-xs text-muted-foreground">Jóvenes</FormLabel>
          <FormControl>

```

```

        <Input
          type="number"
          min={0}
          {...field}
          onChange={(e) => {
            field.onChange(parseInt(e.target.value) || 0);
            recalcTotal();
          }}
          data-testid="input-attendees-youth"
        />
      </FormControl>
    </FormItem>
  )}
/>

{/* Niños */}
<FormField
  control={eventForm.control}
  name="attendeesChildren"
  render={({ field }) => (
    <FormItem>
      <FormLabel className="text-xs text-muted-foreground">Niños</FormLabel>
      <FormControl>
        <Input
          type="number"
          min={0}
          {...field}
          onChange={(e) => {
            field.onChange(parseInt(e.target.value) || 0);
            recalcTotal();
          }}
          data-testid="input-attendees-children"
        />
      </FormControl>
    </FormItem>
  )}
/>

{/* Total - solo lectura, calculado */}
<div className="space-y-1">
  <FormLabel className="text-xs text-muted-foreground">Total (automático)</Fo
  <div className="flex h-10 items-center rounded-md border border-input bg-mu
    {(eventForm.watch("attendeesAdults") || 0) +
      (eventForm.watch("attendeesYouth") || 0) +
      (eventForm.watch("attendeesChildren") || 0)}
    <span className="ml-1 text-xs text-muted-foreground font-normal">personas
  </div>

```

```

    </div>

  </div>
</div>

```

Función `recalcTotal` :

Usar un `useEffect` que mantenga `attendees` sincronizado con la suma:

```

// Dentro del componente, usar watch para calcular automáticamente:
const adults   = eventForm.watch("attendeesAdults")   || 0;
const youth    = eventForm.watch("attendeesYouth")    || 0;
const children = eventForm.watch("attendeesChildren") || 0;

useEffect(() => {
  eventForm.setValue("attendees", adults + youth + children || 1);
}, [adults, youth, children]);

```

Aplicar el mismo patrón en el **form de edición** (`editForm`).

E — Al enviar el form

El payload que se envía al backend debe incluir los cuatro campos:

```

createEventMutation.mutate({
  ...data,
  attendees: adults + youth + children || 1,
  attendeesAdults: adults,
  attendeesYouth: youth,
  attendeesChildren: children,
});

```

F — Mostrar desglose en el detalle del evento

En el tab “Detalles” del `EventDetail` dialog, reemplazar la línea de asistentes:

```

// Antes:
<span className="font-medium">Asistentes:</span>
<span>{selectedEvent.attendees}</span>

// Después:
<span className="font-medium">Asistentes:</span>
<span>
  {selectedEvent.attendees} total

```

```

    {(selectedEvent.attendeesAdults || selectedEvent.attendeesYouth || selectedEvent.attendeesTotal) > 0}
    <span className="text-xs text-muted-foreground ml-1">
      ({selectedEvent.attendeesAdults} ad. / {selectedEvent.attendeesYouth} jóv.
    </span>
  ) : null}
</span>

```

CAMBIO 2 — Hoja de Función: fecha con día de la semana

En `server/eventPdfs.ts`, función `generateHojaFuncionPdf`, modificar el helper `fmtDate` para incluir el día de la semana:

```

// Antes:
const fmtDate = (d: string) => {
  const [y, m, day] = d.split("-");
  return `${day}/${m}/${y}`;
};

// Después:
const DIAS = ["Domingo", "Lunes", "Martes", "Miércoles", "Jueves", "Viernes", "Sábado"];

const fmtDate = (d: string) => {
  const [y, m, day] = d.split("-");
  const fecha = new Date(parseInt(y), parseInt(m) - 1, parseInt(day));
  const diaSemana = DIAS[fecha.getDay()];
  return `${diaSemana} ${day}/${m}/${y}`;
};

```

Resultado en el PDF: Viernes 18/04/2026 en lugar de 18/04/2026 .

Si el evento tiene rango de fechas (varios días), mostrar ambos días:

```

// En el row de fecha:
row(
  "Fecha:",
  event.startDate === event.endDate
    ? fmtDate(event.startDate)
    : `${fmtDate(event.startDate)} al ${fmtDate(event.endDate)} `,
  "Asistentes:",
  desglosarAsistentes(event) // ver helper abajo
);

// Helper para mostrar el desglose en el PDF:

```

```
function desglosarAsistentes(event: EventWithDetails): string {
  const total = event.attendees || 0;
  const adults = (event as any).attendeesAdults || 0;
  const youth = (event as any).attendeesYouth || 0;
  const children = (event as any).attendeesChildren || 0;

  if (adults || youth || children) {
    const partes = [];
    if (adults) partes.push(`${adults} ad.`);
    if (youth) partes.push(`${youth} jóv.`);
    if (children) partes.push(`${children} niños`);
    return `${total} (${partes.join(" / ")})`;
  }
  return String(total);
}
```

CAMBIO 3 — Campos de coordinación por área en el form de evento

A — Nuevas columnas en la DB

```
ALTER TABLE events
  ADD COLUMN IF NOT EXISTS notas_armado      text,
  ADD COLUMN IF NOT EXISTS notas_cocina      text,
  ADD COLUMN IF NOT EXISTS notas_mantenimiento text,
  ADD COLUMN IF NOT EXISTS notas_housekeeping text;
```

B — Schema Drizzle

```
notasArmado:      text("notas_armado"),
notasCocina:      text("notas_cocina"),
notasMantenimiento: text("notas_mantenimiento"),
notasHousekeeping: text("notas_housekeeping"),
```

C — Schema Zod del form

```
notasArmado:      z.string().optional(),
notasCocina:      z.string().optional(),
notasMantenimiento: z.string().optional(),
notasHousekeeping: z.string().optional(),
```

D — En el form de nuevo evento y edición

Agregar una sección debajo del campo "Notas" existente:

```
    { /* Separador */ }
    <div className="col-span-2">
      <p className="text-sm font-medium text-muted-foreground border-t pt-3 mt-1">
        Coordinación por área (opcional)
      </p>
    </div>

    { /* Armado */ }
    <FormField
      control={eventForm.control}
      name="notasArmado"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Armado</FormLabel>
          <FormControl>
            <Textarea
              placeholder="Instrucciones para el equipo de armado..."
              rows={2}
              {...field}
              data-testid="input-notas-armado"
            />
          </FormControl>
        </FormItem>
      ) }
    />

    { /* Cocina */ }
    <FormField
      control={eventForm.control}
      name="notasCocina"
      render={({ field }) => (
        <FormItem>
          <FormLabel>Cocina</FormLabel>
          <FormControl>
            <Textarea
              placeholder="Instrucciones para cocina..."
              rows={2}
              {...field}
              data-testid="input-notas-cocina"
            />
          </FormControl>
        </FormItem>
      ) }
    />
```

```

    })
  />

  { /* Mantenimiento */ }
  <FormField
    control={eventForm.control}
    name="notasMantenimiento"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Mantenimiento</FormLabel>
        <FormControl>
          <Textarea
            placeholder="Instrucciones para mantenimiento..."
            rows={2}
            {...field}
            data-testid="input-notas-mantenimiento"
          />
        </FormControl>
      </FormItem>
    ) }
  />

  { /* Housekeeping */ }
  <FormField
    control={eventForm.control}
    name="notasHousekeeping"
    render={({ field }) => (
      <FormItem>
        <FormLabel>Housekeeping</FormLabel>
        <FormControl>
          <Textarea
            placeholder="Instrucciones para housekeeping..."
            rows={2}
            {...field}
            data-testid="input-notas-housekeeping"
          />
        </FormControl>
      </FormItem>
    ) }
  />

```

E — En el detalle del evento (tab “Detalles”)

Mostrar las notas de área solo si tienen contenido:

```
{(selectedEvent.notasArmado || selectedEvent.notasCocina ||
```

```

selectedEvent.notasMantenimiento || selectedEvent.notasHousekeeping) && (
<div className="col-span-2 space-y-2 pt-2 border-t">
  <p className="text-sm font-semibold">Coordinación por área</p>
  <div className="grid grid-cols-2 gap-3">
    {[
      { label: "Armado",          value: selectedEvent.notasArmado },
      { label: "Cocina",          value: selectedEvent.notasCocina },
      { label: "Mantenimiento", value: selectedEvent.notasMantenimiento },
      { label: "Housekeeping",   value: selectedEvent.notasHousekeeping },
    ]
    .filter((a) => a.value)
    .map((area) => (
      <div key={area.label} className="rounded-md bg-muted p-2">
        <p className="text-xs font-semibold text-muted-foreground mb-1">{area
        <p className="text-sm whitespace-pre-wrap">{area.value}</p>
      </div>
    )))
  </div>
</div>
)}

```

F — En la Hoja de Función PDF

Agregar una sección de coordinación por área debajo de las notas generales:

```

// En generateHojaFuncionPdf, después de la sección de notas:
const areas = [
  { label: "ARMADO",          value: (event as any).notasArmado },
  { label: "COCINA",          value: (event as any).notasCocina },
  { label: "MANTENIMIENTO", value: (event as any).notasMantenimiento },
  { label: "HOUSEKEEPING",   value: (event as any).notasHousekeeping },
].filter((a) => a.value);

if (areas.length > 0) {
  doc.moveDown(0.5);
  doc.moveTo(50, doc.y).lineTo(545, doc.y).strokeColor("#cccccc").stroke();
  doc.moveDown(0.5);
  doc.fontSize(11).font("Helvetica-Bold").text("Coordinación por Área");
  doc.moveDown(0.4);

  for (const area of areas) {
    doc.fontSize(10).font("Helvetica-Bold").text(area.label + ":");
    doc.fontSize(9).font("Helvetica").text(area.value!, { indent: 10, width: 485
    doc.moveDown(0.5);
  }
}

```

Layout resultante en la Hoja de Función:

COORDINACIÓN POR ÁREA

ARMADO:

Colocar mesas redondas, mantel blanco, centros de mesa con flores. Confirmar con decoradora a las 15hs.

COCINA:

Menú: entrada fría, lomo con guarnición, postre.
80 adultos / 20 jóvenes / 10 niños (menú infantil).

MANTENIMIENTO:

Verificar AC del Solárium. Iluminación 100%.

HOUSEKEEPING:

Limpieza profunda del salón antes de las 16hs.

NOTAS

1. Los campos de área (`notasArmado` , `notasCocina` , etc.) son todos opcionales. Si ninguno tiene contenido, no aparecen en el form, en el detalle ni en el PDF.
2. El campo `attendees` sigue existiendo en la DB como el total — es el que se usa en toda la lógica de capacidad y en la Confirmación de Evento para el cliente. Los tres subcampos son informativos y para la Hoja de Función.
3. En la **Confirmación de Evento** (PDF para el cliente), mostrar el desglose también:

Cantidad de personas: 110 total (80 adultos / 20 jóvenes / 10 niños)

4. Aplicar los mismos cambios del form de nuevo evento al **form de edición** (`editForm` + `onSubmitEdit`).
5. El `useEffect` que recalcula el total debe incluir los tres campos en el array de dependencias para que React lo actualice en tiempo real al cambiar cualquiera de los tres inputs.